Pertemuan 3

PHP OOP Fundamental

Class • Object • Property • Method • Constructor • Instantiation

Dari hasil analisis kebutuhan → menjadi program yang dapat dijalankan.

Hilman Lesmana, S.T., M.S.F.

Target Pembelajaran Hari Ini

Setelah sesi ini, mahasiswa tidak hanya mengenal istilah—tetapi mampu membuat program OOP sederhana.

TUJUAN

01 Membuat class & object

Dari kandidat class hasil analisis.

02 Mendefinisikan property & method

Mewakili data dan perilaku object.

03 Menggunakan constructor

Menginisialisasi object saat dibuat.

04 Menjalankan program

Kode berjalan dan hasilnya dapat diamati.

Output utama: 1 program PHP OOP sederhana + source code yang dikumpulkan.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Jembatan dari Pertemuan 2 ke Pertemuan 3

Kita tidak memulai dari nol—kita mengubah hasil analisis menjadi kode.

REKAP

Masalah	→	Actor	→	Kebutuhan	→	Kandidat Class	→	Kode PHP
Sistem peminjaman buku		Petugas & Anggota		Kelola buku + pinjam		Buku, Anggota, Peminjaman		class → object → program

Prinsip: class adalah rancangan; object adalah instance nyata dari rancangan tersebut.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

PHP sebagai Bahasa Implementasi

Pada RPS, fundamental OOP diimplementasikan menggunakan PHP sebelum masuk ke Laravel.

KONTEKS

Yang kita butuhkan hari ini

- PHP terpasang dan bisa dipanggil dari terminal
- Editor kode: Visual Studio Code / IDE
- File latihan berekstensi .php
- Terminal untuk menjalankan program

```
php -v
php praktikum.php
```

Checkpoint setup → mahasiswa harus bisa melihat versi PHP dan menjalankan satu file .php.

Acuan perangkat lunak RPS: PHP 8.5, Visual Studio Code/IDE.

Mental Model: Class vs Object

Gunakan analogi “cetakan” dan “hasil jadi” sebelum melihat sintaks.

KONSEP

CLASS

Blueprint / rancangan

- Mendefinisikan property
- Mendefinisikan method
- Menentukan bentuk object

↓ new ↓

OBJECT

Instance / wujud nyata

- Memiliki nilai property sendiri
- Dapat menjalankan method
- Dapat berinteraksi dengan object lain

Satu class dapat menghasilkan banyak object.

Anatomi Class PHP

Baca kode dari atas ke bawah: nama class → property → method → constructor.

SINTAKS

```php
<?php
class Buku {
    public string $judul;
    public string $penulis;

    public function __construct(string $judul, string $penulis) {
        $this->judul = $judul;
        $this->penulis = $penulis;
    }

    public function tampilkanInfo(): void {
        echo "$this->judul - $this->penulis";
    }
}
```

`class Buku`

Nama class

`public string $judul`

Property

`__construct(...)`

Constructor

`tampilkanInfo()`

Method / behavior

`$this->judul`

Property milik object saat ini

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Property = Data / State

Property menyimpan keadaan object pada suatu waktu.

PROPERTY

STATE OBJECT

Data yang dimiliki object

Buku #1

judul = "OOP Dasar"
penulis = "Budi"

Buku #2

judul = "Laravel"
penulis = "Sari"

Class menentukan property apa yang tersedia; tiap object menyimpan nilai yang bisa berbeda.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Method = Behavior / Perilaku

Method menjelaskan apa yang dapat dilakukan object.

METHOD

Object: Buku

`tampilkanInfo()`	menampilkan informasi
`pinjam()`	mengubah status menjadi dipinjam
`kembalikan()`	mengubah status menjadi tersedia

```
public function pinjam(): void {
    $this->tersedia = false;
}
```

Method boleh membaca atau mengubah state object sesuai tanggung jawabnya.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Constructor: Menyiapkan Object

Constructor dipanggil otomatis ketika object dibuat dengan new.

CONSTRUCTOR

```
class Buku {
    public string $judul;
    public string $penulis;

    public function __construct(string $judul, string
$penulis) {
        $this->judul = $judul;
        $this->penulis = $penulis;
    }
}
```

```
$buku1 = new Buku(
    "OOP Dasar",
    "Budi"
);
```

Apa yang terjadi?

- PHP membuat object baru dari class Buku
- __construct() dijalankan otomatis
- Nilai judul & penulis langsung terisi

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Instantiation: Satu Class, Banyak Object

Operator new menghasilkan instance berbeda dari class yang sama.

OBJECT

```
$buku1 = new Buku("OOP Dasar", "Budi");
$buku2 = new Buku("Laravel", "Sari");
$buku3 = new Buku("UML", "Andi");
```

$buku1	$buku2	$buku3
OOP Dasar	**Laravel**	**UML**
Budi	Sari	Andi
object	*object*	*object*

Class = satu rancangan. Object = banyak instance dengan state masing-masing.

$this: Mengakses Data Milik Object Saat Ini

Pada method dan constructor, $this merujuk ke instance yang sedang bekerja.

$THIS

```
class Buku {
    public string $judul;

    public function tampilkanJudul(): void {
        echo $this->judul;
    }
}
```

Bayangkan pemanggilan:

```
$buku1->tampilkanJudul();
// $this = $buku1

$buku2->tampilkanJudul();
// $this = $buku2
```

$this bukan nama object tertentu. Nilainya mengikuti object yang sedang memanggil method.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Contoh Utuh: Class Buku

Kita gabungkan class, property, constructor, method, object, dan output.

DEMO

```php
<?php

class Buku {
    public string $judul;
    public string $penulis;
    public bool $tersedia = true;

    public function __construct(string $judul, string $penulis) {
        $this->judul = $judul;
        $this->penulis = $penulis;
    }

    public function tampilkanInfo(): void {
        echo "$this->judul - $this->penulis";
    }
}

$buku = new Buku("OOP Dasar", "Budi");
$buku->tampilkanInfo();
```

Struktur berpikir

Class Buku

↓

Object $buku

↓

Method tampilkanInfo()

OOP mulai terlihat ketika data dan perilaku menjadi satu kesatuan.

↓

Output

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Tracing Program: Ikuti Alur Eksekusi

Kemampuan debugging dimulai dari memahami urutan kejadian dalam program.

TRACE

1	**class Buku dibaca**	Belum ada object
2	**new Buku(...)**	Object dibuat
3	**constructor berjalan**	Property diisi
4	**$buku->tampilkanInfo()**	Method dipanggil
5	**echo**	Output muncul

Pertanyaan cepat: pada tahap mana constructor dipanggil?

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Praktikum: Dari Analisis → PHP OOP

Kasus: Sistem Peminjaman Buku. Kerjakan bertahap, jangan langsung mengetik semua kode.

PRAKTIKUM

A Pilih class

Gunakan class Buku

B Tentukan property

judul, penulis, tersedia

C Tentukan method

tampilkanInfo() dan pinjam()

D Buat constructor

Isi judul & penulis

E Buat 2 object

Data berbeda

F Jalankan

Tampilkan hasil

Aturan praktikum: program harus berjalan. Bila error, baca pesan error sebelum bertanya.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Praktikum Terbimbing: Class Buku

Gunakan kerangka ini, lalu mahasiswa melanjutkan sendiri.

KODE

```php
<?php
class Buku {
    public string $judul;
    public string $penulis;
    public bool $tersedia = true;

    public function __construct(string $judul, string $penulis) {
        // TODO: isi property
    }

    public function tampilkanInfo(): void {
        // TODO: tampilkan info buku
    }

    public function pinjam(): void {
        // TODO: ubah status tersedia
    }
}
```

Checkpoint

- Class bernama Buku
- 3 property bertipe jelas
- Constructor mengisi judul & penulis
- Method dapat dipanggil tanpa error

Jangan copy–paste final solution. Lengkapi TODO dengan logika sendiri.

Challenge: Tambahkan Perilaku

Setelah kerangka berjalan, mahasiswa memperluas sendiri.

CHALLENGE

Tugas tantangan di kelas

- Buat method pinjam()
- Jika tersedia = true → ubah menjadi false
- Jika tersedia = false → tampilkan pesan “Buku sedang dipinjam”
- Buat method kembalikan() untuk mengubahnya kembali

Uji dengan urutan

```
$buku->tampilkanInfo();
$buku->pinjam();
$buku->pinjam();
$buku->kembalikan();
```

Amati perubahan state setelah tiap method.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Kesalahan Umum & Cara Membacanya

Biasakan mahasiswa membaca error sebelum meminta solusi.

DEBUG

Class tidak ditemukan	Nama class/file atau penulisan tidak sesuai
Property tidak ada	Cek nama property dan penggunaan $this->
Call to undefined method	Nama method salah / belum dibuat
Parse error	Periksa kurung, titik koma, atau struktur kode

Urutan debugging: baca error → cari baris → cek nama/sintaks → perbaiki → jalankan ulang.

Unjuk Kerja: 3 Menit per Mahasiswa/Pair

Tidak hanya “program jalan”, mahasiswa harus mampu menjelaskan struktur kodenya.

EVALUASI

1. Class apa yang kamu buat?
2. Property apa yang dimiliki object?
3. Method apa yang mewakili behavior?
4. Kapan constructor dijalankan?
5. Berapa object yang kamu buat dan apa bedanya?

Kriteria RPS: program berjalan, struktur class sesuai rancangan, dan kode terbaca.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Tugas Rumah: Ubah 1 Class Menjadi Program

Tugas individu untuk memperkuat konsep sebelum masuk encapsulation di pertemuan 4.

HOMEWORK

Instruksi

- Pilih satu kandidat class dari tugas Pertemuan 2.
- Buat 1 class PHP lengkap dengan minimal 3 property dan 2 method.
- Gunakan constructor untuk mengisi data utama.
- Buat minimal 2 object dengan data berbeda.
- Jalankan program dan simpan bukti output.

Kumpulkan

- .php source code
- Screenshot/hasil output
- Ringkasan 5–8 kalimat: class, property, method, constructor, object

Nama file: NIM_Nama_PBO_M3.php

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Rubrik Singkat Tugas / Unjuk Kerja

Rubrik dibuat konsisten dengan indikator minggu 3 pada RPS.

RUBRIK

Program berjalan	40%	Tidak error saat dijalankan; output sesuai.
Struktur class	25%	Class, property, method, constructor sesuai rancangan.
Pemahaman konsep	20%	Mampu menjelaskan object & instantiation.
Keterbacaan kode	15%	Penamaan jelas, indentasi rapi, komentar seperlunya.

Fokus penilaian bukan panjang kode, tetapi ketepatan penerapan konsep.

Acuan: RPS Pemrograman Berorientasi Objek TI033015, Minggu 3.

Penutup: Dari Object ke Desain yang Lebih Baik

Pertemuan 4 akan membawa kita ke encapsulation, access modifier, dan code review.

NEXT

Minggu 2	→	Minggu 3	→	Minggu 4	→	Berikutnya
Analisis kebutuhan		Class & Object		Encapsulation		Advanced OOP → Laravel

Pertanyaan sebelum selesai: “Apa yang membuat class kita mudah diubah tanpa merusak kode lain?”

Jawabannya akan kita mulai pada Pertemuan 4.